receive no response, the node that sent the query can remove the destination node from the corresponding k-bucket.

Join

As in Pastry, a node that needs to join the network needs to know at least one other node. The joining node sends its identifier to the node as though it is a key to be found. The response it receives allows the new node to create its k-buckets.

Leave or Fail

When a node leaves the network or fails, other nodes update their k-buckets using the concurrent process described before.

2.4.6 A Popular P2P Network: BitTorrent

BitTorrent is a P2P protocol, designed by Bram Cohen, for sharing a large file among a set of peers. However, the term *sharing* in this context is different from other file-sharing protocols. Instead of one peer allowing another peer to download the whole file, a group of peers takes part in the process to give all peers in the group a copy of the file. File sharing is done in a collaborating process called a *torrent*. Each peer participating in a torrent downloads chunks of the large file from another peer that has it and uploads chunks of that file to other peers that do not have it, a kind of *tit-for-tat*, a trading game played by kids. The set of all peers that takes part in a torrent is referred to as a *swarm*. A peer in a swarm that has the complete content file is called a *seed*; a peer that has only part of the file and wants to download the rest is called a *leech*. In other words, a swarm is a combination of seeds and leeches. BitTorrent has gone through several versions and implementations. We first describe the original one, which uses a central node called a *tracker*. We then show how some new versions eliminate the tracker by using DHT.

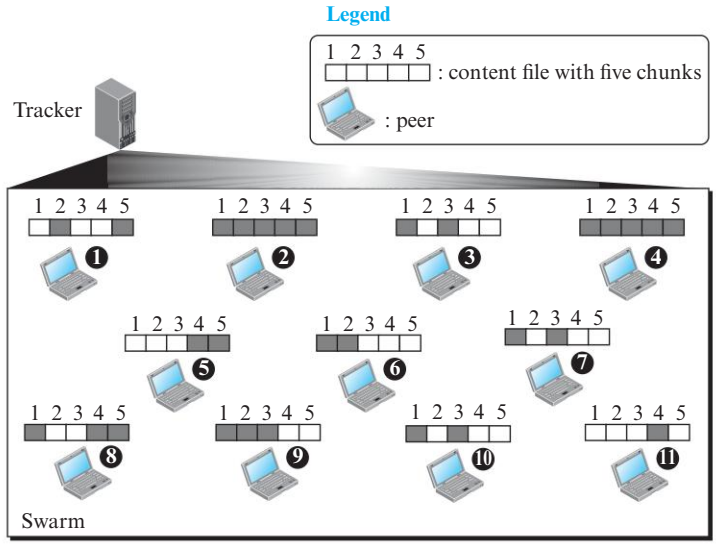
BitTorrent with A Tracker

In the original BitTorrent, there is another entity in a torrent, called the tracker, which, as the name implies, tracks the operation of the swarm, as described later. Figure 2.57 shows an example of a torrent with seeds, leeches, and the tracker.

In the figure, the file to be shared, the content file, is divided into five pieces (chunks). Peers 2 and 4 already have all the pieces; other peers have some pieces. The pieces that each peer has are shaded. Uploading and downloading of the pieces will continue. Some peers may leave the torrent; some new peers may join the torrent.

Now assume a new peer wants to download the same content file. The new peer accesses the BitTorrent server with the name of the content file. It receives a metafile, named the torrent file, that contains the information about the pieces in the content file and the address of the tracker that handles that specific torrent. The new peer now accesses the tracker and receives the addresses of some peers in the torrent, normally called *neighbors*. The new peer is now part of the torrent and can download and upload pieces of the content file. When it has all the pieces, it may leave the torrent or remain in the torrent to help other peers, including the new peers that have joined after it, to get all pieces of the content file. Nothing can prevent a peer from leaving the torrent before it has all the pieces and joining later or not joining again.

Figure 2.57 *Example of a torrent*



Note: Peers 2 and 4 are seeds; others are leeches.

Although the process of joining, sharing, and leaving a torrent looks simple, the BitTorrent protocol applies a set of policies to provide fairness, to encourage the peers to exchange pieces with other peers, to prevent overloading a peer with requests from other peers, and to allow a peer to find peers that provide better service.

To avoid overloading a peer and to achieve fairness, each peer needs to limit its concurrent connection to a number of neighbors; the typical value is four. A peer flags a neighbor as unchoked or choked. It also flags them as interested or uninterested.

In other words, a peer divides its lists of neighbors into two distinct groups: *unchoked* and *choked*. It also divides them into *interested* and *uninterested* groups. The unchoked group is the list of peers that the current peer has concurrently connected to; it continuously uploads and downloads pieces from this group. The choked group is the list of neighbors that the peer is not currently connected to but may connect to in the future.

Every 10 seconds, the current peer tries a peer in the interested but choked group for a better data rate. If this new peer has a better rate than any of the unchoked peers, the new peer may become unchoked, and the peer with the lowest data rate in the unchoked group may move to the choked group. In this way, the peers in the unchoked group always have the highest data rate among those peers probed. Using this strategy divides the neighbors into subgroups in which those neighbors with compatible data transfer rates will communicate with each other. The idea of tit-for-tat trading strategy described above can be seen in this policy.

To allow a newly joined peer, which does not yet have a piece to share, to also receive pieces from other peers, every 30 seconds a peer randomly promotes a single interested peer, regardless of its uploading rate, from the choked group and flags it as unchoked. This action is called *optimistic unchoking*.

The BitTorrent protocol tries to provide a balance between the number of pieces each peer may have at each moment by using a strategy called the *rarest-first*. Using this strategy, a peer tries to first download the pieces with the fewest repeated copies among the neighbors. In this way, these pieces are circulated faster.

Trackerless BitTorrent

In the original BitTorrent design, if the tracker fails, new peers cannot connect to the network and updating is interrupted. There are several implementations of BitTorrent that eliminate the need for a centralized tracker. In the implementation that we describe here, the protocol still uses the tracker, but not a central one. The job of tracking is distributed among some nodes in the network. In this section, we show how Kademlia DHT can be used to achieve this goal, but we avoid becoming involved in the details of a specific protocol.

In BitTorrent with a central tracker, the job of the tracker is to provide the list of peers in a swarm when given a metadata file that defines the torrent. If we think of the hash function of metadata as the key and the hash function of the list of peers in a swarm as the value, we can let some nodes in a P2P network play the role of trackers. A new peer that joins the torrent sends the hash function of the metadata (key) to the node that it knows. The P2P network uses Kademlia protocol to find the node responsible for the key. The responsible node sends the *value*, which is actually the list of peers in the corresponding torrent, to the joining peer. Now the joining peer can use the BitTorrent protocol to share the content file with peers in the list.

2.5 SOCKET INTERFACE PROGRAMMING

In Section 2.2, we discussed the principle of the client-server paradigm. In Section 2.3, we discussed some standard applications using this paradigm. In this section, we show how to write some simple client-server programs using C, a procedural programming language. We chose the C language in this section for two reasons. First, socket programming traditionally started in the C language. Second, the low-level feature of the C language better reveals some subtleties in this type of programming. In Chapter 11, we expand this idea in Java, which provides a more compact version. However, this section can be skipped without loss of continuity in the study of the book.

2.5.1 Socket Interface in C

We discussed socket interface in Section 2.2. In this section, we show how this interface is implemented in the C language. The important issue in socket interface is to understand the role of a socket in communication. The socket has no buffer to store data to be sent or received. It is capable of neither sending nor receiving data. The socket just acts as a reference or a label. The buffers and necessary variables are created inside the operating system.

Data Structure for Socket

The C language defines a socket as a structure (struct). The socket structure is made of five fields; each socket address itself is a structure made of five fields, as shown in Figure 2.58. Note that the programmer should not redefine this structure; it is already defined in the header files. We briefly discuss the five fields in a socket structure.